

Owner Name: taniya2013

## Practice TEST

**This sheet is a part of my AI ML Bootcamp conducted by CodroidHub Private Limited**

1. What is the difference between "==" and "is" in Python? Give one example where they produce different results.

Ans- The difference is that == compares values (equality) while is compares memory addresses (identity)

2 Why does  $0.1 + 0.2 == 0.3$  return False ? What is the correct way to compare floats?

Ans-  $0.1 + 0.2 == 0.3$  is False because computers store decimals as binary fractions, creating tiny rounding errors (like 0.30000000000000004) the correct way to compare them is by checking if they are "close enough" using `math.isclose()`

3 Explain the LEGB rule. In what order does Python resolve a variable name?

Ans- The LEGB rule is the exact blueprint Python uses to look up variable names. Python doesn't search randomly, it checks four specific scope levels one after the other. Local -> Enclosing -> Global -> Built-in.

4 What is a mutable default argument bug? How do you fix it?

Ans- The mutable default argument bug is a famous Python trap that happens when we use a mutable object—like a list [] or a dictionary {}—as a default value for a function argument. we can fix it - instead of creating a fresh, empty list every time you call the function. Python reuses the exact same list across every single function call.

5 What does `and` return when all values are truthy? What does `or` return when all values are falsy?

Ans- when all values are truthy `and` will return last value of it . in case of `or` when values gives falsy result it will give the last

value as result .

6 What is the difference between "//" and "/" ? What does -7 // 2 return and why?

Ans- "//" is known as floor division it gives nearest smaller integer value, "/" is a division operator , it does normal division and result gives in decimal (float) value.

7 What does for-else do in Python? When does the else block run?

Ans - In Python, the else block of a for loop runs only when the loop finishes without encountering a break statement.

8 Why should you never use str += inside a loop? What should you use instead?

Ans- Strings in Python are immutable, which means they cannot be changed after they are created. When you use str += inside a loop, Python creates a new string object every time, which makes the program slow and inefficient for large amounts of text.

9 What is the difference between a generator and a list comprehension?

Ans - A list comprehension creates and stores all elements in memory at once and returns a list. A generator produces elements one by one when needed and returns a generator object, making it more memory efficient.

10 What does @functools.wraps(func) do, and why is it mandatory in decorators?

Ans- @functools.wraps(func) is used inside a decorator to preserve the original function's metadata, such as:

1)Function name (name) 2)Docstring (doc) 3)Module information

1. What does type(True) return? \_\_\_\_\_ My ans- show its class=< class 'bool'>

2. Is bool a subclass of int in Python? \_\_\_\_\_ my ans- yes Yes, bool is a subclass of int in Python. Therefore, True behaves like 1 and False behaves like 0 in numeric operations.

3. What is the output of True + True + True ? \_\_\_\_\_

My ans- 3

4. What Python version introduced the walrus operator `:=` ? \_\_\_\_\_

My ans- python3.8

5. What Python version introduced match-case ? \_\_\_\_\_

My ans- python 3.10

6. What is the CPython integer cache range? \_\_\_\_\_

My ans= -5 to 256

7. What does `id()` return? \_\_\_\_\_

My ans= The unique identity (memory address) of an object

8. Name three falsy values in Python. \_\_\_\_\_

My ans= False, 0, "" (empty string)

9. What operator checks membership in a collection? \_\_\_\_\_

My ans= in

10. What does `**kwargs` capture in a function? \_\_\_\_\_

My ans- Variable-length keyword arguments (as a dictionary)

```
In [ ]: #Fill in the blanks
        # 1. To compare floats correctly:
        abs(a - b) < _____
```

Answer: 1e-9

Reason/Observation: ven though mathematically  $0.1 + 0.2 = 0.3$ , Python stores slightly different values.

Therefore, instead of checking: `a == b` we check whether the difference between them is very small: `abs(a - b) < 1e-9`

where  $1e-9$  (0.000000001) is a tiny tolerance called epsilon. If the difference is smaller than epsilon  $\rightarrow$  treat the numbers as equal. If the difference is larger than epsilon  $\rightarrow$  treat them as different. So the blank is:  $\text{abs}(a - b) < 1e-9$

In [ ]: . To swap two variables **in** one line:

a, b = \_\_\_\_\_, \_\_\_\_\_

Answer - a, b = b, a

Reason:- The value of b is assigned to a. The value of a is assigned to b. Python performs both assignments simultaneously, so no temporary variable is needed.

In [ ]: *# 3. To check if a variable is None (idiomatic way):*

**if** result \_\_\_\_\_ **None**:

Answer:- "is"

Reason:- None is a special singleton object in Python. The idiomatic (recommended) way to check for "None" is using "is" or "is not", "not ==".

In [ ]: *# 4. The correct fix for mutable default argument:*

```
def append_item(item, lst=____):  
    if lst is None:  
        lst=[]
```

```
-----  
NameError                                Traceback (most recent call last)  
/tmp/ipykernel_8356/3237713432.py in <cell line: 0>()  
      1 # 4. The correct fix for mutable default argument:  
----> 2 def append_item(item, lst=____):  
      3     if lst is None:  
      4         lst=[]  
      5 ans= lst=None
```

**NameError**: name '\_\_\_\_' is not defined

Answer:- lst=None

Reason:- Using a mutable object like "[]" as a default argument is dangerous because the same list is reused across function calls.

```
In [ ]: # 5. To build a string from a List efficiently:  
result = _____.join(words)
```

Answer= result = "".join(words)

Reason:- Strings are immutable in Python. Repeatedly using + to concatenate strings creates many temporary strings and is inefficient.

Using join() is faster and more memory-efficient

```
In [ ]: # 6. The walrus operator in a while loop:  
while _____ := f.read(1024):  
    process(chunk)
```

Answer:- while chunk := f.read(1024): process(chunk)

Reason:- The walrus operator (:=) assigns a value to a variable and returns that value in the same expression "f.read(1024)" reads up to 1024 characters. The result is assigned to "chunk". If "chunk" is not empty, the loop continues. When end-of-file is reached, "f.read()" returns an empty string (""), which is False, so the loop stops.

```
In [ ]: # 7. To preserve function metadata in a decorator:  
@functools._____(func)  
def wrapper(*args, **kwargs):
```

Answer= @functools.wraps(func)

def wrapper(\*args, \*\*kwargs):

Reason:- functools.wraps() preserves the original function's metadata such as:

**"name"**

**"doc"**

**"module"**

**"annotations"**

Without wraps, the decorated function appears as wrapper instead of the original function name.

```
In [ ]: # 8.  
        Floor division of -9 by 2 equals: _____
```

Answer:-5

Reason:- Python's floor division operator // rounds down toward negative infinity.

```
In [ ]: # 9. To get both index and value in a loop:  
        for i, v in _____(my_list):
```

Answer:- for i, v in enumerate(my\_list):

Reason:- "enumerate()" returns both the index and the value of each element while looping through a list.

```
In [ ]: # 10.  
        'and' returns the _____ falsy value, or the _____ value if all truthy.  
  
        File "/tmp/ipykernel_8356/596598400.py", line 2  
            'and' returns the _____ falsy value, or the _____ value if all truthy.  
                          ^  
SyntaxError: invalid syntax
```

Answer:- 'and' returns the first falsy value, or the last value if all truthy.

Reason:- The and operator evaluates operands from left to right:

Returns the first falsy value it encounters. If all values are truthy, it returns the last value.

```
In [ ]: # E1 - Your prediction: _____  
        x = 10  
        y = 3  
        print(x // y)
```

```
3
1
3.3333333333333335
```

Answer= 3

```
In [ ]: print(x % y)
```

```
-----
NameError                                Traceback (most recent call last)
/tmp/ipykernel_1370/3096063612.py in <cell line: 0>()
----> 1 print(x % y)
```

NameError: name 'x' is not defined

Answer - 1

```
In [ ]: print(x / y)
```

Answer- 3.3333333333333335

```
In [ ]: # E2 - Your prediction: _____
name = " Alice "
print(name.strip().upper().replace('A', '@'))
```

@LICE

Answer:- @LICE

```
In [ ]: # E3 - Your prediction: _____
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

[1, 2, 3, 4]

Answer:- [1, 2, 3, 4]

```
In [ ]: # E4 - Your prediction: _____
print(bool(""), bool(" "), bool(0), bool([0]), bool(None))
```

False True False True False

Answer:- False True False True False

```
In [ ]: # I1 - Your prediction: _____  
print(0 and "hello") # Your prediction: _____
```

Answer:- false

Reason= In python 0 is taken as false so, when we use and operator we should have both values true to get output as true here first is already false so the ans is false

```
In [ ]: print("" or "default") # Your prediction: _____
```

Answer:- default

Reason = "" is an empty string And in python empty string taken as false and non empty string taken as true so result is taken as default as or operator is used .

```
In [ ]: print(None or [] or 0) #Your prediction: _____
```

Answer- 0

Reason=The or operator returns the first truthy value it finds. If all values are falsy, it returns the last value.

```
In [ ]: print("a" and "b" and "c") # Your prediction: _____
```

Answer:- c

Reason= a,b,c are 3 strings here a,b,c gives true so result is taken as c as last element.

```
In [ ]: # I2 - Your prediction: _____  
x = 5  
result = "positive" if x > 0 else "negative" if x < 0 else "zero"  
print(result)  
  
y = -3  
result2 = "positive" if y > 0 else "negative" if y < 0 else "zero"
```



```
print(result2)
my ans = positive
reason= because value of y is less than 0 so result is taken as negative.
```

positive  
negative

Answer- positive

Reason= because value of x is greater than 0 so result is taken as positive

Answer- positive

Reason= because value of y is less than 0 so result is taken as negative.

```
In [ ]: # I3 - Your prediction: _____
s = "Hello, World!"
print(s[7:12])
```

World  
!dlroW ,olleH  
Hlo ol!

Answer- World

```
In [ ]: print(s[::-1])
```

Answer- !dlroW ,olleH

```
In [ ]: print(s[::2])
```

Answer- Hlo ol!

```
In [ ]: # I4 - Your prediction: _____
def f(x, multiplier=2):
    return x * multiplier
print(f(3))
```

Answer- 6

```
In [ ]: print(f(3, 3))
```

Answer- 9

```
In [ ]: print(f(multiplier=5, x=4))
```

Answer- 20

```
In [ ]: # T1 – Your prediction: _____  
print(-7 // 2)
```

Answer:- -4

```
In [ ]: print(-7 % 2)
```

Answer- -1

```
In [ ]: print(7 // -2)
```

Answer- -4

```
In [ ]: print(7 % -2)
```

Answer- -1

```
In [ ]: # T2 – Your prediction: _____  
print(2 ** 3 ** 2) # right-associative exponentiation
```

Answer- 512

```
In [ ]: print((2 ** 3) ** 2)
```

Answer- 64

```
In [ ]: # T3 – Your prediction: _____  
def add_tag(item, tags=[]):  
    tags.append(item)
```

```
return tags
print(add_tag("python"))
```

Answer- ['python']

```
In [ ]: print(add_tag("ml"))
my ans= ['python', 'ml']
```

Answer- ['python', 'ml']

```
In [ ]: print(add_tag("ai"))
```

Answer- ['python', 'ml', 'ai']

```
In [ ]: # T4 - Your prediction: _____
funcs = []
for i in range(3):
    funcs.append(lambda: i)
print([f() for f in funcs])
```

[0]

[1, 1]

[2, 2, 2]

Answer-[0]

[1, 1]

[2, 2, 2]

```
In [ ]: # H1 - Integer caching: Your prediction: _____
a = 256
b = 256
print(a is b)
```

True

Answer- True

```
In [ ]: c = 257
```

```
d = 257
print(c is d)
```

Answer- True

```
In [ ]: # H2 - Boolean arithmetic: Your prediction: _____
print(True + True)
```

Answer- 2

Reason- True = 1 in python so True+True=2

```
In [ ]: print(True * 10)
```

Answer-10

Reason- True = 1 in python so True\*10=10

```
In [ ]: print(sum([True, False, True, True, False]))
```

Answer- 3

Reason- True = 1 in python so sum([True, False, True, True, False])=1 + 0 + 1 + 1 + 0 = 3

```
In [ ]: print(int(True), int(False))
```

Answer- 1 0

Reason- True = 1 in python so int(True)=1 and int(False)=0

```
In [ ]: # H3 - for-else: Your prediction: _____
for n in [2, 4, 6, 8]:
    if n % 2 != 0:
        print("Found odd")
        break
    else:
        print("All even")
```

```
All even
All even
All even
All even
All even
Found odd
```

Answer- All even

```
In [ ]: for n in [2, 3, 6, 8]:
        if n % 2 != 0:
            print("Found odd")
            break
        else:
            print("All even")
```

Answer- Found odd

```
In [ ]: #SYN-1: Find and fix all syntax errors
        # DON'T RUN yet - find the bugs by reading first
        def greet(name
        message = f"Hello, {name}!"
        return message
```

Answer- def greet(name): message = f"Hello, {name}!" return message

```
In [ ]: for i in range(5)
        if i % 2 == 0:
            print(i, "is even")
```

Answer- for i in range(5): if i % 2 == 0: print(i, "is even")

```
In [ ]: def calculate_fee(amount):
        fee = amount * 2 / 100
        if fee < 5:
            fee == 5 # BUG: What is wrong here?
        if fee > 200:
            fee == 200 # BUG: And here?
        return fee
        print(calculate_fee(100)) # Expected: 5 (minimum applies)
```

5

Answer-

```
def calculate_fee(amount):
```

```
    fee= amount * 2 / 100
```

```
    if fee < 5:
```

```
        fee = 5
```

```
    if fee > 200:
```

```
        fee = 200
```

```
    return fee
```

```
print(calculate_fee(100))
```

```
In [ ]: print(calculate_fee(5000))
```

100.0

Answer- 100.0

```
In [ ]: print(calculate_fee(15000))
```

200

Answer- 200

```
In [ ]: def classify_grade(score):  
        if score >= 90:  
            grade = "A"  
        elif score >= 80:  
            grade = "B"  
        elif score >= 70:  
            grade = "C"  
        elif score >= 60:
```

```
grade = "D"  
grade = "F" # BUG: Where is the error?  
return grade  
print(classify_grade(95))
```

A

Answer- A

EXPLANATION= def classify\_grade(score):

if score >= 90:

    grade = "A"

elif score >= 80:

    grade = "B"

elif score >= 70:

    grade = "C"

elif score >= 60:

    grade = "D"

else:

    grade = "F"

return grade

print(classify\_grade(95))

```
In [ ]: print(classify_grade(75))
```

C

Answer- C

```
In [ ]: print(classify_grade(45))
```

F

Answer- F

```
In [ ]: # LOG-3: Count vowels function – returns wrong count
def count_vowels(text):
    count = 0
    for char in text:
        if char in "aeiou": # BUG: What's missing?
            count += 1
    return count
print(count_vowels("Hello World"))
```

3

Answer- 3 Explanation=

```
def count_vowels(text):
```

```
    count = 0
```

```
    for char in text:
```

```
        if char in "aeiouAEIOU": # BUG: What's missing?
```

```
            count += 1
```

```
    return count
```

```
    print(count_vowels("Hello World"))
```

```
In [ ]: print(count_vowels("APPLE"))
```



2

Answer- 2

```
In [ ]: def divide(a, b):  
        return a / b  
results = [divide(10, x) for x in [2, 5, 0, 1]]  
print(results)
```

[5.0, 2.0, None, 10.0]

Answer- ZeroDivisionError

fix code=

```
def divide(a, b):
```

```
    if b == 0:
```

```
        return None
```

```
    return a / b
```

```
results = [divide(10, x) for x in [2, 5, 0, 1]]
```

```
print(results)
```

```
In [ ]: data = {"name": "Alice", "age": 25}  
print(f"Name: {data['name']}, Score: {data['score']}")
```

Name: Alice, Score: 0

Answer- key KeyError

fix code=

```
data = {"name": "Alice", "age": 25}
```

```
print(f"Name: {data['name']}, Score: {data.get('score', 0)}")
```

```
In [ ]: counter = 0
def increment():
    counter += 1 # BUG: Why does this fail?
    return counter
increment()
```

1

my ans= UnboundLocalError

fix=

```
def increment(counter):
```

```
    counter += 1
```

```
    return counter
```

```
counter = 0
```

```
counter = increment(counter)
```

```
print(counter)
```

```
In [ ]: # HIDDEN-1: This runs without error but produces WRONG results
# Intent: Remove all negative numbers from the list
numbers = [1, -2, 3, -4, 5, -6, 7]
for num in numbers:
    if num < 0:
        numbers.remove(num)
print(numbers)
```

[1, 3, 5, 7]

Answer-

```
numbers = [1, -2, 3, -4, 5, -6, 7]
```

```
for num in numbers[:]:
```

```
    if num < 0:

        numbers.remove(num)

print(numbers)
```

```
In [ ]: multipliers = []
        for i in range(3):
            multipliers.append(lambda x: x * i)
            print([f(10) for f in multipliers])
```

```
[0, 10, 20]
```

Answer-

```
multipliers = []
```

```
for i in range(3):
```

```
    multipliers.append(lambda x, i=i: x * i)
```

```
print([f(10) for f in multipliers])
```

```
In [ ]: def average(numbers):
        return sum(numbers) / len(numbers)
        # Test cases to handle:
        average([])
```

```

-----
ZeroDivisionError                                Traceback (most recent call last)
/tmp/ipykernel_3401/3528340848.py in <cell line: 0>()
      2     return sum(numbers) / len(numbers)
      3 # Test cases to handle:
----> 4 average([])

/tmp/ipykernel_3401/3528340848.py in average(numbers)
      1 def average(numbers):
----> 2     return sum(numbers) / len(numbers)
      3 # Test cases to handle:
      4 average([])

```

ZeroDivisionError: division by zero

my ans=

ZeroDivisionError

Fix code=

```
def average(numbers):
```

```
    if len(numbers) == 0:
```

```
        return None
```

```
    return sum(numbers) / len(numbers)
```

```
In [ ]: average([5])
```

```
Out[ ]: 5.0
```

Answer- 5.0

```
In [ ]: average(None)
```

```
-----  
TypeError                                Traceback (most recent call last)  
/tmp/ipykernel_3401/352041246.py in <cell line: 0>()  
----> 1 average(None)
```

```
/tmp/ipykernel_3401/3528340848.py in average(numbers)  
    1 def average(numbers):  
----> 2     return sum(numbers) / len(numbers)  
    3 # Test cases to handle:  
    4 average([])
```

TypeError: 'NoneType' object is not iterable

Answer- TypeError

fix=

```
def average(numbers):  
  
    if numbers is None:  
  
        raise TypeError("numbers cannot be None")  
  
    if len(numbers) == 0:  
  
        raise ValueError("Cannot calculate average of an  
        empty list")  
  
    return sum(numbers) / len(numbers)
```

```
In [ ]: average(["a", "b"])
```

Answer- TypeError

fix=

```
def average(numbers):
```

```

    if numbers is None:

        raise TypeError("numbers cannot be None")

    if not isinstance(numbers, (list, tuple)):

        raise TypeError("numbers must be a list or tuple")

    if len(numbers) == 0:

        raise ValueError("Cannot calculate average of an empty list")

    if not all(isinstance(x, (int, float)) for x in numbers):

        raise TypeError("All elements must be numbers")

    return sum(numbers) / len(numbers)

```

In [ ]: `''' average([1, 2, None, 4])`

2.3333333333333335

Answer- NameError

fix=""

```

def average(numbers):

    numbers = [x for x in numbers if x is not None]

    return sum(numbers) / len(numbers)

print(average([1, 2, None, 4]))

```

In [ ]: `def is_valid_password(pwd):  
# Test these boundaries:  
# "" → False (empty)`

```
# "abc" → False (too short)
# "a" * 21 → False (too long)
# "Abcdefg1!" → True
# "ABCDEFG1!" → False (no lowercase)
# "abcdefg1!" → False (no uppercase)
# "Abcdefgh!" → False (no digit)
# "Abcdefg12" → False (no special char)
pass
```

False

Answer=

```
def is_valid_password(pwd):
    if not isinstance(pwd, str):
        return False

    if len(pwd) < 8 or len(pwd) > 20:
        return False

    has_upper = any(c.isupper() for c in pwd)
    has_lower = any(c.islower() for c in pwd)
    has_digit = any(c.isdigit() for c in pwd)
    has_special = any(not c.isalnum() for c in pwd)

    return has_upper and has_lower and has_digit and has_special

print(is_valid_password(""))
```

```
In [ ]: print(is_valid_password("abc"))
```

False

Answer- False

```
In [ ]: print(is_valid_password("a" * 21))
```

False

Answer- False

```
In [ ]: print(is_valid_password("Abcdefg1!"))
```

True

Answer- True

```
In [ ]: print(is_valid_password("ABCDEFG1!"))
```

False

Answer- False

```
In [ ]: print(is_valid_password("abcdefg1!"))
```

False

Answer- False

```
In [ ]: print(is_valid_password("Abcdefgh!"))
```

False

Answer- False

```
In [ ]: print(is_valid_password("Abcdefg12"))
```

False

Answer - False

```
In [ ]: def validate_student_age(age):  
        return 5 <= age <= 18  
        validate_student_age("fifteen")
```

*# validate\_student\_age(None) → TypeError  
# validate\_student\_age(15.7) → handle float (round or reject?)  
# validate\_student\_age("15") → handle string number (convert or reject?)*



-----  
TypeError Traceback (most recent call last)

/tmp/ipykernel\_287/2437267192.py in <cell line: 0>()

```
1 def validate_student_age(age):  
2     return 5 <= age <= 18  
----> 3 validate_student_age("fifteen")  
4 '''my ans= TypeError  
5 fix=
```

/tmp/ipykernel\_287/2437267192.py in validate\_student\_age(age)

```
1 def validate_student_age(age):  
----> 2     return 5 <= age <= 18  
3 validate_student_age("fifteen")  
4 '''my ans= TypeError  
5 fix=
```

TypeError: '<=' not supported between instances of 'int' and 'str'

Answer- TypeError

fix=

```
def validate_student_age(age):  
  
    if not isinstance(age, (int, float)):  
  
        return False  
  
    return 5 <= age <= 18
```

In [ ]: print(validate\_student\_age("fifteen"))

Answer- TypeError

In [ ]: validate\_student\_age(-1)

Out[ ]: False

Answer- False

```
In [ ]: from typing_extensions import TYPE_CHECKING
        validate_student_age(None)
```

```
-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_287/3131406873.py in <cell line: 0>()
----> 1 validate_student_age(None)

/tmp/ipykernel_287/2437267192.py in validate_student_age(age)
      1 def validate_student_age(age):
----> 2     return 5 <= age <= 18
      3 validate_student_age("fifteen")
      4 '''my ans= TypeError
      5 fix=
```

TypeError: '<=' not supported between instances of 'int' and 'NoneType'

Answer- TypeError

```
In [ ]: validate_student_age(15.7)
```

```
Out[ ]: True
```

Answer- True

```
In [ ]: validate_student_age("15")
```

```
Out[ ]: True
```

Answer- TypeError

fix=

```
def validate_student_age(age):

    try:

        age = int(age)

    except (ValueError, TypeError):
```

```
    return False
```

```
    return 5 <= age <= 18
```

```
In [ ]: validate_student_age("15")
```

Answer - True

```
In [ ]: def find_duplicates_slow(items):
        duplicates = []
        for i in range(len(items)):
            for j in range(i + 1, len(items)):
                if items[i] == items[j] and items[i] not in duplicates:
                    duplicates.append(items[i])
        return duplicates
# Questions:
# 1. What is the time complexity of this function? O(____)
# 2. What is the space complexity? O(____)
# 3. At 1 million items, roughly how many comparisons? ____
# 4. Rewrite it with O(n) time using a set/dict:
```

Answer1 -Outer loop = n times

inner loop = almost n times

To total comparisons:

$n \times n = n^2$

2

$O(n^2)$

Answer2 - duplicate list will store duplicate elements in worst case.

Answer3 - Comparisons  $\approx n(n-1)/2$  if  $n = 1,000,000$

$1000000 \times 999999 / 2 = 499,999,500,000$

≈ 500 billion comparisons

```
In [ ]: #using set:
def find_duplicates(items):
    seen = set()
    duplicates = set()

    for item in items:
        if item in seen:
            duplicates.add(item)
        else:
            seen.add(item)

    return list(duplicates)
print(find_duplicates([1, 2, 3, 2, 4, 1, 5]))
```

[1, 2]

Answer- [1,2]

```
In [ ]: #using dict:
def find_duplicates(items):
    count = {}
    duplicates = []

    for item in items:
        count[item] = count.get(item, 0) + 1

    for item, freq in count.items():
        if freq > 1:
            duplicates.append(item)

    return duplicates
print(find_duplicates([1, 2, 3, 2, 4, 1, 5]))
```

[1, 2]

Answer- [1,2]

```
In [ ]: def calculate_upi_fee(amount):
        """
```

```
Calculate UPI transaction fee.  
Returns: fee as float, or None if input is invalid.  
""  
# Your implementation here  
pass
```

Answer-

```
def calculate_upi_fee(amount):  
  
    if not isinstance(amount, int):  
        return None  
  
    if amount <= 0:  
        return None  
  
    if amount < 100:  
        return "free"  
  
    fee = amount * 0.018  
  
    if fee < 2:  
        fee = 2.0  
    elif fee > 1000:  
        fee = 1000.0  
  
    return float(fee)
```

```
In [ ]: # Test your implementation:  
test_cases = [  
    (50, "free")  
]  
for amount, expected in test_cases:  
    result = calculate_upi_fee(amount)  
    status = " " if result == expected or (expected is None and result is None) else " "  
    print(f"{status} calculate_upi_fee({amount}) = {result} | expected: {expected}")  
  
calculate_upi_fee(50) = free | expected: free
```

Answer- calculate\_upi\_fee(50) = free | expected: free

```
In [ ]: test_cases = [  
        (100, 2.0)  
    ]  
    for amount, expected in test_cases:  
        result = calculate_upi_fee(amount)  
        status = " " if result == expected or (expected is None and result is None) else " "  
    print(f"{status} calculate_upi_fee({amount}) = {result} | expected: {expected}")  
  
    calculate_upi_fee(100) = 2.0 | expected: 2.0  
  
    Answer- calculate_upi_fee(100) = 2.0 | expected: 2.0
```

```
In [ ]: test_cases = [  
        (500, 9.0)  
    ]  
    for amount, expected in test_cases:  
        result = calculate_upi_fee(amount)  
        status = " " if result == expected or (expected is None and result is None) else " "  
    print(f"{status} calculate_upi_fee({amount}) = {result} | expected: {expected}")  
  
    calculate_upi_fee(500) = 9.0 | expected: 9.0  
  
    Answer- calculate_upi_fee(500) = 9.0 | expected: 9.0
```

```
In [ ]: test_cases = [  
        (70000, 1000.0)  
    ]  
    for amount, expected in test_cases:  
        result = calculate_upi_fee(amount)  
        status = " " if result == expected or (expected is None and result is None) else " "  
    print(f"{status} calculate_upi_fee({amount}) = {result} | expected: {expected}")  
  
    calculate_upi_fee(70000) = 1000.0 | expected: 1000.0  
  
    Answer- calculate_upi_fee(70000) = 1000.0 | expected: 1000.0
```

```
In [ ]: test_cases = [  
        (499.99, None)  
    ]  
    for amount, expected in test_cases:  
        result = calculate_upi_fee(amount)  
        status = " " if result == expected or (expected is None and result is None) else " "  
    print(f"{status} calculate_upi_fee({amount}) = {result} | expected: {expected}")
```

calculate\_upi\_fee(499.99) = None | expected: None

Answer- calculate\_upi\_fee(499.99) = None | expected: None

```
In [ ]: test_cases = [  
        ("500", None)  
    ]  
    for amount, expected in test_cases:  
        result = calculate_upi_fee(amount)  
        status = " " if result == expected or (expected is None and result is None) else " "  
    print(f"{status} calculate_upi_fee({amount}) = {result} | expected: {expected}")
```

calculate\_upi\_fee(500) = None | expected: None

Answer- calculate\_upi\_fee(500) = None | expected: None

```
In [ ]: test_cases = [  
        (-100, None)  
    ]  
    for amount, expected in test_cases:  
        result = calculate_upi_fee(amount)  
        status = " " if result == expected or (expected is None and result is None) else " "  
    print(f"{status} calculate_upi_fee({amount}) = {result} | expected: {expected}")
```

calculate\_upi\_fee(-100) = None | expected: None

Answer- calculate\_upi\_fee(-100) = None | expected: None

```
In [ ]: test_cases = [  
        (0, None)  
    ]
```

```

]
for amount, expected in test_cases:
    result = calculate_upi_fee(amount)
    status = " " if result == expected or (expected is None and result is None) else " "
print(f"{status} calculate_upi_fee({amount}) = {result} | expected: {expected}")

```

calculate\_upi\_fee(0) = None | expected: None

Answer- calculate\_upi\_fee(0) = None | expected: None

```

In [ ]: def calculate_bmi(weight_kg, height_m):
        """ Calculate BMI and classify health risk.
        Returns dict with bmi, category, risk_level – or None if invalid."""
        pass
        # Test cases:
        print(calculate_bmi(70, 1.75)) # Normal: ~22.9
        print(calculate_bmi(50, 1.75)) # Underweight: ~16.3
        print(calculate_bmi(100, 1.75)) # Obese: ~32.7
        print(calculate_bmi(-5, 1.75)) # Invalid
        print(calculate_bmi(70, 0)) # Invalid (division by zero risk)
        print(calculate_bmi("seventy", 1.75))

```

None

None

None

None

None

None

Answer- {'bmi': 22.9, 'category': 'Normal', 'risk\_level': 'Average'}

{'bmi': 16.3, 'category': 'Underweight', 'risk\_level': 'Low'}

{'bmi': 32.7, 'category': 'Obese', 'risk\_level': 'High'}

In [ ]: Fix code-

```

def calculate_bmi(weight_kg, height_m):
    """Calculate BMI and classify health risk.
    Returns dict with bmi, category, risk_level – or None if invalid.
    """

```



```

# Validate input types
if not isinstance(weight_kg, (int, float)) or not isinstance(height_m, (int, float)):
    return None

# Validate positive values
if weight_kg <= 0 or height_m <= 0:
    return None

bmi = weight_kg / (height_m ** 2)
bmi = round(bmi, 1)

if bmi < 18.5:
    category = "Underweight"
    risk_level = "Low"
elif bmi < 25:
    category = "Normal"
    risk_level = "Average"
elif bmi < 30:
    category = "Overweight"
    risk_level = "Increased"
else:
    category = "Obese"
    risk_level = "High"

return {
    "bmi": bmi,
    "category": category,
    "risk_level": risk_level
}

# Test cases
print(calculate_bmi(70, 1.75))
print(calculate_bmi(50, 1.75))
print(calculate_bmi(100, 1.75))
print(calculate_bmi(-5, 1.75))
print(calculate_bmi(70, 0))
print(calculate_bmi("seventy", 1.75))

```

In [ ]: print(calculate\_bmi(50, 1.75))

```
{'bmi': 16.3, 'category': 'Underweight', 'risk_level': 'Low'}
```

Answer- {'bmi': 16.3, 'category': 'Underweight', 'risk\_level': 'Low'}

```
In [ ]: print(calculate_bmi(100, 1.75))
```

```
{'bmi': 32.7, 'category': 'Obese', 'risk_level': 'High'}
```

Answer- {'bmi': 32.7, 'category': 'Obese', 'risk\_level': 'High'}

```
In [ ]: print(calculate_bmi(-5, 1.75))
```

None

Answer- None

```
In [ ]: print(calculate_bmi(70, 0))
```

None

Answer- None

```
In [ ]: print(calculate_bmi("seventy", 1.75))
```

None

Answer- None

```
In [ ]: def apply_discount(price, coupon_code):
```

```
    pass
```

```
    # Test cases:
```

```
    print(apply_discount(1000, "SAVE10")) # 10% off = ₹900
```

```
    print(apply_discount(1000, "save20")) # case-insensitive = ₹800
```

```
    print(apply_discount(600, "FLAT50")) # ₹50 off = ₹550
```

```
    print(apply_discount(300, "FLAT50")) # Invalid: order not > ₹500
```

```
    print(apply_discount(100, "SAVE10")) # Invalid: below minimum ₹200
```

```
    print(apply_discount(1000, "BOGUS"))
```

```
{'original': 1000, 'discount': 100.0, 'final': 900.0, 'message': 'Coupon applied successfully'}
```

Answer-

```
def apply_discount(price, coupon_code):
```

```

if not isinstance(price, (int, float)) or price <= 0:
    return None

    coupon_code = coupon_code.upper()

if coupon_code == "SAVE10":
    if price < 200:
        return {"message": "Minimum order ₹200 required"}
        discount = price * 0.10

elif coupon_code == "SAVE20":
    if price < 200:
        return {"message": "Minimum order ₹200 required"}
        discount = price * 0.20

elif coupon_code == "FLAT50":
    if price <= 500:
        return {"message": "Order must be greater than ₹500"}
        discount = 50

else:
    return {"message": "Invalid coupon code"}

return {
    "original": price,
    "discount": round(discount, 2),
    "final": round(price - discount, 2),
    "message": "Coupon applied successfully"
}
print(apply_discount(1000, "SAVE10"))

```

```

In [ ]: print(apply_discount(1000, "save20"))
        my ans=

```

```
{'original': 1000, 'discount': 200.0, 'final': 800.0, 'message': 'Coupon applied successfully'}
```

Answer- {'original': 1000, 'discount': 200.0, 'final': 800.0, 'message': 'Coupon applied successfully'}

```

In [ ]: print(apply_discount(600, "FLAT50"))

```

```
{'original': 600, 'discount': 50, 'final': 550, 'message': 'Coupon applied successfully'}
```

Answer- {'original': 600, 'discount': 50, 'final': 550, 'message': 'Coupon applied successfully'}

```
In [ ]: print(apply_discount(300, "FLAT50"))
```

```
{'message': 'Order must be greater than ₹500'}
```

Answer- {'message': 'Order must be greater than ₹500'}

```
In [ ]: METRO_CITIES = {"mumbai", "delhi", "bangalore", "chennai", "kolkata", "hyderabad"}
def calculate_net_salary(basic_monthly, city="non-metro"):
    """
    Calculate net monthly salary after deductions.
    Returns dict with breakdown of all components.
    """
    pass
# Test:
print(calculate_net_salary(50000, "mumbai")) # Metro HRA
print(calculate_net_salary(50000, "Pune")) # Non-metro HRA
print(calculate_net_salary(10000)) # Below PF threshold
```

```
{'basic': 50000, 'hra': 25000.0, 'pf': 6000.0, 'gross_salary': 75000.0, 'net_salary': 69000.0}
```

my ans=

```
METRO_CITIES = {
    "mumbai", "delhi", "bangalore",
    "chennai", "kolkata", "hyderabad"
}
```

```
def calculate_net_salary(basic_monthly, city="non-metro"):
    if not isinstance(basic_monthly, (int, float)) or basic_monthly <= 0:
        return None
```

```
    city = city.lower()
```

```
    # HRA
```

```
    if city in METRO_CITIES:
        hra = basic_monthly * 0.50
    else:
```

```

        hra = basic_monthly * 0.40

# PF
    if basic_monthly >= 15000:
        pf = basic_monthly * 0.12
    else:
        pf = 0

    gross_salary = basic_monthly + hra
    net_salary = gross_salary - pf

    return {
        "basic": basic_monthly,
        "hra": hra,
        "pf": pf,
        "gross_salary": gross_salary,
        "net_salary": net_salary
    }
print(calculate_net_salary(50000, "mumbai"))

```

Answer- {'basic': 50000, 'hra': 25000.0, 'pf': 7500.0, 'gross\_salary': 75000.0, 'net\_salary': 67500.0}

```
In [ ]: print(calculate_net_salary(50000, "Pune"))
```

```
{'basic': 50000, 'hra': 20000.0, 'pf': 6000.0, 'gross_salary': 70000.0, 'net_salary': 64000.0}
```

Answer- {'basic': 50000, 'hra': 20000.0, 'pf': 7500.0, 'gross\_salary': 77500.0, 'net\_salary': 69250.0}

```
In [ ]: print(calculate_net_salary(10000))
```

```
{'basic': 10000, 'hra': 4000.0, 'pf': 0, 'gross_salary': 14000.0, 'net_salary': 14000.0}
```

my ans= {'basic': 10000, 'hra': 4000.0, 'pf': 0, 'gross\_salary': 14000.0, 'net\_salary': 14000.0}

```
In [ ]: # B1: Write a function that takes a temperature in Celsius and returns it in
# Fahrenheit AND Kelvin. Return as a tuple: (fahrenheit, kelvin)
# Formula: F = (C × 9/5) + 32 | K = C + 273.15
def convert_temperature(celsius):
    pass
# Tests:
```

```
    assert convert_temperature(0) == (32.0, 273.15)
    assert convert_temperature(100) == (212.0, 373.15)
print("B1 passed ")
```

B1 passed

Answer-

B1 passed

```
In [ ]: #B2: Write a function that checks if a string is a palindrome
# Ignore case and spaces
# "A man a plan a canal Panama" → True
# "hello" → False
def is_palindrome(text):
    pass
# Tests:
    assert is_palindrome("racecar") == True
    assert is_palindrome("hello") == False
    assert is_palindrome("A man a plan a canal Panama") == True
    assert is_palindrome("") == True
print("B2 passed ")
```

B2 passed

Answer- B2 Passed

```
In [ ]: fizzbuzz = [
    "FizzBuzz" if i % 15 == 0
    else "Fizz" if i % 3 == 0
    else "Buzz" if i % 5 == 0
    else i
    for i in range(1, 31)
]
assert fizzbuzz[2] == "Fizz" # 3
assert fizzbuzz[4] == "Buzz" # 5
assert fizzbuzz[14] == "FizzBuzz" #15
assert fizzbuzz[6] == 7
print("B3 passed ", fizzbuzz[:15])
```

B3 passed [1, 2, 'Fizz', 4, 'Buzz', 'Fizz', 7, 8, 'Fizz', 'Buzz', 11, 'Fizz', 13, 14, 'FizzBuzz']

Answer- B3 passed [1, 2, 'Fizz', 4, 'Buzz', 'Fizz', 7, 8, 'Fizz', 'Buzz', 11, 'Fizz', 13, 14, 'FizzBuzz']

```
In [ ]: # — INTERMEDIATE —————
# I1: Write a decorator `timer` that measures and prints execution time
# Use functools.wraps. The print format: "<func_name> took X.XXXs"
import functools
import time

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()

        print(f"{func.__name__} took {end - start:.4f}s")
        return result

    return wrapper

@timer
def slow_sum(n):
    """Sum numbers from 0 to n."""
    return sum(range(n))

result = slow_sum(10_000_000)

print(f"Result: {result}")
print(f"Function name preserved: {slow_sum.__name__}")
```

slow\_sum took 0.1119s

Result: 49999995000000

Function name preserved: slow\_sum

Answer-

slow\_sum took 0.1119s Result: 49999995000000 Function name preserved: slow\_sum

```
In [ ]: # I2: Implement a retry decorator
```

```

# retry(max_attempts=3, delay=0) - retries function on exception
# After max_attempts, raise the last exception
def retry(max_attempts=3, delay=0):
    passIn
    attempt_count = 0
    @retry(max_attempts=3)
    def flaky_api_call():
        global attempt_count
        attempt_count += 1
        if attempt_count < 3:
            raise ConnectionError("API timeout")
        return "success"
    print(flaky_api_call()) # Should succeed on 3rd attempt
    print(f"Took {attempt_count} attempts")

```

success

Took 4 attempts

Answer-

success Took 4 attempts

```

In [ ]: # I3: Generator for Fibonacci sequence
# Write a generator function fibonacci(n) that yields the first n Fibonacci numbers
# Then use it to: find the first Fibonacci number > 1000
def fibonacci(n):
    a, b = 0, 1

    for _ in range(n):
        yield a
        a, b = b, a + b
fib_list = list(fibonacci(10))
assert fib_list == [0, 1, 1, 2, 3, 5, 8, 13, 21, 34], f"Got {fib_list}"
first_over_1000 = next(x for x in fibonacci(100) if x > 1000)

print(f"First Fibonacci > 1000: {first_over_1000}")
print("I3 passed")

```

First Fibonacci > 1000: 1597

I3 passed

Answer-



First Fibonacci > 1000: 1597 I3 passed

```
In [ ]: # A1: Implement a memoization decorator from scratch (without functools.lru_cache)
# The decorator should cache function results based on arguments
# Add a .cache_info() method that returns
def memoize(func):
    cache = {}
    hits = 0
    misses = 0
    def wrapper(*args):
        nonlocal hits, misses
        if args in cache:
            hits += 1
            return cache[args]
        misses += 1
        result = func(*args)
        cache[args] = result
        return result
    def cache_info():
        return {
            "hits": hits,
            "misses": misses,
            "size": len(cache)
        }
    wrapper.cache_info = cache_info
    return wrapper
@memoize
def fib(n):
    if n < 2:
        return n
    return fib(n - 1) + fib(n - 2)
print(fib(30))      # Should be fast
print(fib.cache_info())
```

832040

{'hits': 28, 'misses': 31, 'size': 31}

Answer-

832040 {'hits': 28, 'misses': 31, 'size': 31}

```
In [ ]: # A2: Implement a pipeline function that chains multiple functions
# pipeline(f1, f2, f3)(x) should be equivalent to f3(f2(f1(x)))
def pipeline(*functions):
    pass
# Test:
process = pipeline(
    str.strip,
    str.lower,
    lambda s: s.replace(" ", "_")
)
assert process(" Hello World ") == "hello_world"
assert process(" Python Programming ") == "python_programming"
print("A2 passed ")
```

A2 passed

Answer- A2 Passed

```
In [ ]: # SCENARIO 1: Student Report Card Generator
# You receive student data as a list of dicts:
# [{"name": "Alice", "marks": {"math": 85, "science": 92, "english": 78}}, ...]
#
# Build a function generate_report(students) that returns a list of dicts with:
# - name, total, average (2 decimal places), grade (A/B/C/D/F), rank (1=highest avg)
# - Handle: missing subjects, marks outside 0-100, empty students list
# - Sort output by rank (highest average first)
students = [
    {"name": "Alice", "marks": {"math": 85, "science": 92, "english": 78}},
    {"name": "Bob", "marks": {"math": 72, "science": 68, "english": 80}},
    {"name": "Charlie", "marks": {"math": 95, "science": 88, "english": 91}},
    {"name": "Priya", "marks": {"math": 60, "science": 55, "english": 70}},
]
def generate_report(students):
    pass
    for record in generate_report(students):
        print(record)
```

```
{'name': 'Charlie', 'total': 274, 'average': 91.33, 'grade': 'A', 'rank': 1}
{'name': 'Alice', 'total': 255, 'average': 85.0, 'grade': 'B', 'rank': 2}
{'name': 'Bob', 'total': 220, 'average': 73.33, 'grade': 'C', 'rank': 3}
{'name': 'Priya', 'total': 185, 'average': 61.67, 'grade': 'D', 'rank': 4}
```

Answer-

```
students = [
{"name": "Alice", "marks": {"math": 85, "science": 92, "english": 78}},
{"name": "Bob", "marks": {"math": 72, "science": 68, "english": 80}},
{"name": "Charlie", "marks": {"math": 95, "science": 88, "english": 91}},
{"name": "Priya", "marks": {"math": 60, "science": 55, "english": 70}},
]

def generate_report(students):
    if not students:
        return []

    report = []

    for student in students:
        name = student.get("name", "Unknown")
        marks = student.get("marks", {})

        # Validate marks
        valid_marks = []
        for subject, mark in marks.items():
            if not (0 <= mark <= 100):
                raise ValueError(f"Invalid mark {mark} in {subject} for {name}")
            valid_marks.append(mark)

        total = sum(valid_marks)
        average = round(total / len(valid_marks), 2) if valid_marks else 0

        # Grade calculation
        if average >= 90:
            grade = "A"
        elif average >= 80:
            grade = "B"
        elif average >= 70:
            grade = "C"
        elif average >= 60:
            grade = "D"
```

```

        else:
            grade = "F"

        report.append({
            "name": name,
            "total": total,
            "average": average,
            "grade": grade
        })

    # Sort by average descending
    report.sort(key=lambda x: x["average"], reverse=True)

    # Assign ranks
    for rank, student in enumerate(report, start=1):
        student["rank"] = rank

    return report

for record in generate_report(students):
    print(record)

```

```

In [ ]: # Given a text string, write analyze_text(text) that returns:
# - word_count: total words
# - unique_words: count of unique words (case-insensitive)
# - top_5_words: List of (word, count) tuples, sorted by frequency desc
# - avg_word_length: average length of all words (rounded to 2 dp)
# - Longest_word: the longest word
# Strip punctuation. Ignore words shorter than 3 characters.
import re
sample_text = """
Python is a versatile programming language. Python is used for web development,
data science, machine learning, and automation. Python code is clean and readable.
Many data scientists use Python for building machine learning models."""
def analyze_text(text):
    pass
result = analyze_text(sample_text)
for key, value in result.items():
    print(f"{key}: {value}")

```

```
word_count: 30
unique_words: 22
top_5_words: [('python', 4), ('for', 2), ('data', 2), ('machine', 2), ('learning', 2)]
avg_word_length: 6.17
longest_word: programming
```

Answer-

```
import re
from collections import Counter

sample_text = """
Python is a versatile programming language. Python is used for web development,
data science, machine learning, and automation. Python code is clean and readable.
Many data scientists use Python for building machine learning models.
"""

def analyze_text(text):
    # Extract words, remove punctuation, convert to lowercase
    words = re.findall(r'\b[a-zA-Z]+\b', text.lower())

    # Ignore words shorter than 3 characters
    words = [word for word in words if len(word) >= 3]

    word_count = len(words)
    unique_words = len(set(words))

    counter = Counter(words)
    top_5_words = counter.most_common(5)

    avg_word_length = round(
        sum(len(word) for word in words) / word_count, 2
    ) if word_count else 0

    longest_word = max(words, key=len) if words else ""

    return {
        "word_count": word_count,
        "unique_words": unique_words,
```

```

        "top_5_words": top_5_words,
        "avg_word_length": avg_word_length,
        "longest_word": longest_word
    }

```

```

result = analyze_text(sample_text)

```

```

for key, value in result.items():
    print(f"{key}: {value}")

```

```

In [ ]: # SCENARIO 3: Multi-step Data Pipeline
#
# You receive raw sales data from a CSV-like string.
# Pipeline steps:
# Step 1: parse_sales_data(raw) → list of dicts
# Step 2: clean_data(records) → remove records with missing/invalid fields
# Step 3: calculate_metrics(records) → total revenue, avg order value, top product
#
# Each step should be a separate function – then compose them.
raw_sales = """order_id,product,quantity,unit_price
1001,Laptop,2,45000
1002,Mouse,5,500
1003,,1,1200
1004,Keyboard,0,800
1005,Monitor,1,18000
1006,Headphones,-1,2500
1007,Laptop,1,45000
"""

def parse_sales_data(raw):
    """Parse CSV string into list of dicts."""
    pass
def clean_data(records):
    """Remove records with missing product or invalid quantity/price."""
    pass
def calculate_metrics(records):
    """Return total_revenue, avg_order_value, top_product."""
    pass
# Run the pipeline:
parsed = parse_sales_data(raw_sales)
cleaned = clean_data(parsed)
metrics = calculate_metrics(cleaned)

```

```
print(f"Parsed: {len(parsed)} records | Cleaned: {len(cleaned)} records")
print(metrics)
```

Parsed: 7 records | Cleaned: 4 records

{'total\_revenue': 155500.0, 'avg\_order\_value': 38875.0, 'top\_product': 'Laptop'}

Answer-

```
raw_sales = """order_id,product,quantity,unit_price
1001,Laptop,2,45000
1002,Mouse,5,500
1003,,1,1200
1004,Keyboard,0,800
1005,Monitor,1,18000
1006,Headphones,-1,2500
1007,Laptop,1,45000
"""
```

```
def parse_sales_data(raw):
    lines = raw.strip().split("\n")
    headers = lines[0].split(",")

    records = []

    for line in lines[1:]:
        values = line.split(",")
        record = dict(zip(headers, values))

        # convert safely
        try:
            record["quantity"] = int(record["quantity"])
        except:
            record["quantity"] = None

        try:
            record["unit_price"] = float(record["unit_price"])
        except:
            record["unit_price"] = None

        records.append(record)
```

```
    return records
```

```
def clean_data(records):
```

```
    cleaned = []
```

```
    for r in records:
```

```
        if (
```

```
            not r.get("product") or
```

```
            r["quantity"] is None or
```

```
            r["unit_price"] is None or
```

```
            r["quantity"] <= 0 or
```

```
            r["unit_price"] <= 0
```

```
        ): 
```

```
            continue
```

```
        cleaned.append(r)
```

```
    return cleaned
```

```
def calculate_metrics(records):
```

```
    if not records:
```

```
        return {
```

```
            "total_revenue": 0,
```

```
            "avg_order_value": 0,
```

```
            "top_product": None
```

```
        }
```

```
total_revenue = 0
```

```
product_sales = {}
```

```
for r in records:
```

```
    revenue = r["quantity"] * r["unit_price"]
```

```
    total_revenue += revenue
```

```
    product = r["product"]
```



```

        product_sales[product] = product_sales.get(product, 0) + revenue

    avg_order_value = total_revenue / len(records)
    top_product = max(product_sales, key=product_sales.get)

    return {
        "total_revenue": total_revenue,
        "avg_order_value": round(avg_order_value, 2),
        "top_product": top_product
    }

# PIPELINE EXECUTION
parsed = parse_sales_data(raw_sales)
cleaned = clean_data(parsed)
metrics = calculate_metrics(cleaned)

print(f"Parsed: {len(parsed)} records | Cleaned: {len(cleaned)} records")
print(metrics)

```

In [ ]: 1. The following function is called 10,000 times per second in production. What would you change?

```

def get_tax_rate(country):
    rates = {"IN": 18, "US": 8, "UK": 20, "DE": 19}
    return rates.get(country, 0)

```

Answer- This function is re-creating the dictionary every time it is called, which is inefficient when it runs 10,000 times/second

code=

```

RATES = {
    "IN": 18,
    "US": 8,
    "UK": 20,
    "DE": 19
}

def get_tax_rate(country):
    return RATES.get(country, 0)

```

Ques 2. A function builds a report for each of 500,000 users. It currently uses a list to accumulate results and appends one item per user. What is the memory concern, and what is the solution?

Answer- If a function builds a report for 500,000 users using a list and stores each result in memory, then:

i)All 500,000 report objects stay in RAM at once ii)Memory usage can become very large (especially if each record is heavy) May cause: i)high memory consumption ii)slow performance iii)possible MemoryError / crash iv)GC (garbage collector) pressure example=

```
def generate_reports(users):
    for user in users:
        yield build_report(user)
    for report in generate_reports(users):
        process(report)
```

In [ ]: 3. What **is** wrong **with** this error handling?

```
try:
    result = process_payment(order)
except:
    pass
```

Answer-

1. Silent failure The error is completely ignored, so: Payment might fail But system will act like everything is fine This can lead to incorrect or fake successful orders.
2. Bugs are hidden Any issue (network error, validation error, server crash) is swallowed. Makes debugging very difficult.
3. No logging or monitoring No record of what went wrong No alerts in production systems
4. Too broad exception handling

except:

This catches everything, including critical exceptions like:

SystemExit KeyboardInterrupt

Ques 4. A function calls an external API to get exchange rates. What reliability measures should you add?

Answer- When a function calls an external API (exchange rates), it becomes unreliable because network and service issues can happen. So we must add resilience and fault-tolerance mechanisms.

1. Timeout handling Prevent hanging requests: `requests.get(url, timeout=3)`
2. Retry mechanism Retry temporary failures: `for i in range(3): try: return fetch_rate() except: continue`
3. Exponential backoff

Wait longer between retries: `1s → 2s → 4s` (prevents API overload) 4. Caching Store recent exchange rates: avoid repeated API calls improves speed + reduces cost Example: cache for 5–10 minutes 5. Fallback mechanism If API fails: use last known value or secondary API provider 6. Proper error handling + logging except Exception as e: `log.error(e)` 7. Rate limit handling respect API limits handle HTTP 429 responses

In [ ]: 5. Rewrite this unreadable one-liner as clean, readable code:  
`r = {k:v for k,v in {n:(lambda x:x*(x+1)//2)(n) for n in range(1,11)}.items() if v%3==0}`

Answer-

```
result = {}

for n in range(1, 11):
    value = n * (n + 1) // 2

    if value % 3 == 0:
        result[n] = value
```

In [ ]: 6. What PEP 8 violations exist in this code?

```
def CalcBMI(W,H):
    BMI=W/(H*H)
    if BMI<18.5: return 'Underweight'
    elif BMI<25: return 'Normal'
    elif BMI<30: return 'Overweight'
    else: return 'Obese'
List violations and rewrite:
ans=
def calc_bmi(weight, height):
    bmi = weight / (height * height)

    if bmi < 18.5:
```

```

        return "Underweight"
    elif bmi < 25:
        return "Normal"
    elif bmi < 30:
        return "Overweight"
    else:
        return "Obese"

```

Answer-

```

def calc_bmi(weight, height):
    bmi = weight / (height * height)

    if bmi < 18.5:
        return "Underweight"
    elif bmi < 25:
        return "Normal"
    elif bmi < 30:
        return "Overweight"
    else:
        return "Obese"

```

```

In [ ]: def f(l):
        r = []
        for i in range(0, len(l)):
            if l[i] > 0:
                r = r + [l[i] * 2]
        return r

# Problems to fix:
# 1. No docstring / type hints
# 2. Bad variable names (l looks like 1)
# 3. r = r + [item] is O(n²) – should use append or comprehension
# 4. range(0, len(l)) – antipattern, just iterate directly
# 5. No edge case handling (None input, non-numeric items)

```

Answer- from typing import List, Optional

```

def double_positive_numbers(nums: Optional[List[int]]) -> List[int]:
    """
    Returns a list of doubled positive numbers from input list.

```

```

"""
if not nums:
    return []

result = []

for num in nums:
    if isinstance(num, (int, float)) and num > 0:
        result.append(num * 2)

return result

```

```

In [ ]: def add_to_cart(item, cart=[]):
        cart.append(item)
        print(f"Cart: {cart}")
        return cart
        # Problems:
        # 1. Mutable default argument – cart persists across calls
        # 2. No input validation
        # 3. No type hints or docstring
        # 4. Mixes side effects (print) with return value
        # Demonstrate the bug first:
        cart1 = add_to_cart("apple")
        cart2 = add_to_cart("banana") # Should start fresh!
        print("Bug: cart1 is cart2:", cart1 is cart2)

```

Answer- from typing import List, Optional

```

def add_to_cart(item: str, cart: Optional[List[str]] = None) -> List[str]:
    """
    Adds an item to shopping cart and returns updated cart.
    """
    if cart is None:
        cart = []

    cart.append(item)
    return cart

```

```

In [ ]: def has_duplicate_slow(items):

```

```

for i in range(len(items)):
    for j in range(i + 1, len(items)):
        if items[i] == items[j]:
            return True
    return False
# Rewrite with O(n) time complexity using a set:
def has_duplicate_fast(items):
    pass
# Verify same results, then time both:
import time
test_data = list(range(10000)) + [9999] # duplicate at end
start = time.time()
has_duplicate_slow(test_data)
print(f"Slow: {time.time() - start:.4f}s")
start = time.time()
has_duplicate_fast(test_data)
print(f"Fast: {time.time() - start:.4f}s")

```

Answer-

```

def has_duplicate_slow(items):
    for i in range(len(items)):
        for j in range(i + 1, len(items)):
            if items[i] == items[j]:
                return True
    return False

```

```

# O(n) solution
def has_duplicate_fast(items):
    seen = set()

    for item in items:
        if item in seen:
            return True
        seen.add(item)

    return False

```

```

# Verify same results, then time both
import time

test_data = list(range(10000)) + [9999] # duplicate at end

start = time.time()
print(has_duplicate_slow(test_data))
print(f"Slow: {time.time() - start:.4f}s")

start = time.time()
print(has_duplicate_fast(test_data))
print(f"Fast: {time.time() - start:.4f}s")

```

```

In [ ]: students = [
    {
        "id": "S001",
        "name": "Arjun Sharma",
        "courses": {
            "python": {"total_lessons": 20, "completed": 15, "quiz_scores": [85, 92,
78]},
            "ml": {"total_lessons": 30, "completed": 12, "quiz_scores": [70, 68]}
        },
        "subscription": "premium",
        "join_date": "2024-01-15"
    },
    # ... 499 more students
]
Phase 1: Analyze Requirements (write your analysis in this cell)
1.What are the core operations needed?

2. What validations are required?

3.What output formats are needed?

```

Answer- We need to process each student and compute learning insights: Per student operations: i)Calculate course completion % - completed / total\_lessons \* 100 ii)Compute average quiz score per course iii)Identify best performing course iv)Identify low engagement courses (low completion or low scores) System-level operations: i)Aggregate stats across all students: a)total students b)premium vs free users c)average completion rate per course d)top performing students ii)Filter students based on criteria: a)high performers b)at-risk learners

Answer2- ans= We must handle messy / real-world data safely: Student-level validation: id must exist and be unique name must not be empty subscription must be valid (free / premium)

Answer3- ans= We likely need structured analytics output, such as:

```
{
  "id": "S001",
  "name": "Arjun Sharma",
  "course_progress": {
    "python": {
      "completion_pct": 75.0,
      "avg_quiz": 85.0
    },
    "ml": {
      "completion_pct": 40.0,
      "avg_quiz": 69.0
    }
  },
  "overall_score": 76.5,
  "risk_level": "medium"
}
{
  "total_students": 500,
  "premium_users": 320,
  "avg_course_completion": {
    "python": 68.2,
    "ml": 55.1
  },
  "top_students": ["S001", "S045"]
}
```



Your analysis: This is a large-scale student learning analytics system, so we first break it into core operations, validations, and outputs

```
In [ ]: Phase 2: Identify Edge Cases
List at least 8 edge cases that your implementation must handle:
1.missing course data
2.Missing fields in course
3.total_lessons = 0
4.completed > total_lessons
5.Empty quiz_scores list
6.Invalid quiz score values
7.Missing student fields
8. Large-scale performance issue (500+ students)
```

```
In [ ]: Answer-
from _typeshed import FileDescriptorLike
Phase 3: Design Solution Approach
Plan your functions before writing code:
Function 1: validate_student(student) → bool
Input:{
    "id": "S001",
    "name": "Arjun Sharma",
    "courses": {...},
    "subscription": "premium",
    "join_date": "2024-01-15"
}
Output:True # if student is valid
False # if student data is invalid
Edge cases handled:
1.missing Fields
{"id": "S001"}
2. Empty or null values
"name": ""
3.Invalid data types
"completed": "ten"
4.Invalid subscription
"subscription": "gold"
5.Empty courses
"courses": {}
6. Invalid course structure
"python": {"completed": 10}
```

```
7. Logical inconsistency
completed > total_lessons
8. Corrupted quiz scores
"quiz_scores": [90, "A", None]
```

```
In [ ]: Function 2: calculate_course_progress(course_data) → dict
Input: {
    "total_lessons": 20,
    "completed": 15,
    "quiz_scores": [85, 92, 78]
}
Output: {
    "completion_pct": 75.0,
    "avg_quiz_score": 85.0,
    "status": "on_track" # or "at_risk" / "not_started"
}
Edge cases handled:
1. Missing fields
{"completed": 10}
2. total_lessons = 0
{"total_lessons": 0}
3. completed > total_lessons
completed = 25, total_lessons = 20
4. Empty quiz_scores
{"quiz_scores": []}
5. Invalid quiz score values
{"quiz_scores": [90, "A", None]}
6. Negative values
completed = -5
7. Missing course completely
None or {}
8. Extremely large datasets (performance)
Many quiz scores
Ensure O(n) only per course
```

```
In [ ]: function 3: generate_student_report(student) → dict
Input: {
    "id": "S001",
    "name": "Arjun Sharma",
    "courses": {
        "python": {...},
```

```
    "ml": {...}
  },
  "subscription": "premium"
}
```

```
Output:{
  "id": "S001",
  "name": "Arjun Sharma",
  "subscription": "premium",
  "course_reports": {
    "python": {
      "completion_pct": 75.0,
      "avg_quiz_score": 85.0,
      "status": "on_track"
    },
    "ml": {
      "completion_pct": 40.0,
      "avg_quiz_score": 69.0,
      "status": "at_risk"
    }
  },
  "overall_progress": 62.5,
  "risk_level": "medium"
}
```

Edge cases handled:

1. Missing student fields

```
{ "id": "S001" }
```

2. Empty courses

```
"courses": {}
```

3. Invalid course data

```
"python": None
```

4. All courses invalid

overall\_progress should be 0

5. Division by zero

No courses → avoid average calculation error

6. Mixed valid + invalid courses

Only valid courses considered in final metrics

7. Missing student name or ID

fallback values:

or skip record (depending system design)

8. Large number of courses

ensure O(n) per student (no nested inefficiency)

```

In [ ]: Function 4: platform_summary(students) → dict
Input: [
  {"id": "S001", "name": "Arjun", "courses": {...}, "subscription": "premium"},
  {"id": "S002", "name": "Neha", "courses": {...}, "subscription": "free"},
  ...
]
Output: {
  "total_students": 500,
  "active_students": 460,
  "subscription_split": {
    "premium": 300,
    "free": 200
  },
  "avg_platform_progress": 63.4,
  "top_performers": ["S001", "S045", "S120"],
  "at_risk_students": ["S078", "S210"]
}
Edge cases handled:
1. Empty student list
students = []

return safe empty structure with zeros

2. Invalid student records
{"id": None}

skip corrupted entries

3. Missing subscription field

default to "free"

4. Missing or broken course data

handled via generate_student_report

5. Division by zero
No valid students → avg_platform_progress = 0

6. Large dataset (500+ students)
Ensure single-pass iteration (O(n))
Avoid nested loops inside loops

7. Duplicate student IDs

```

```
either:  
override latest  
or ignore duplicates (design decision)  
  8. All students invalid  
  
return:  
empty top_performers  
empty at_risk list  
safe defaults
```